

Programmierungsmethodik I & Programmiertechnik

Andreas P. Priesnitz

`andreas.priesnitz@haw-hamburg.de`

Hochschule für Angewandte Wissenschaften Hamburg
Fakultät Informatik und Digitale Gesellschaft

Sommersemester 2026

Allgemeines

Modulzuordnung: B.Sc. Informatik Technischer Systeme

Programmierungsmethodik I (PM1) 1. Sem. 6 CP

Programmiertechnik (PT) 1. Sem. 6 CP

Aufwand:

12 CP $\sim$ 360 h $\sim$ 20 h/W $\cdot$ 13 W + 80 h (Prüfung etc.)
 $\rightsquigarrow$ 8 h LV + 12 h Eigenarbeit pro Sem.woche

Hinweis: Zeitvorgabe ist realistisch, insb. wenn keine Vorkenntnisse!

Informationen

Material & Kommunikation in **Moodle**:

<http://moodle.haw-hamburg.de/> (Zugriffscod: Prog4U)

Moodle-**Forum Organisation** für Information zu Abläufen

Hinweis: bei Fragen bitte erst in Moodle nachschauen, dann melden

Literatur

ergänzende Literatur (ähnlich, nicht gleich zu dieser Veranstaltung):

- Ullenboom, C.: **Java ist auch eine Insel**, 18. Auflage, Rheinwerk, 2025.
- Goll, J.; Heinisch, C.: **Java als erste Programmiersprache**, 8. Auflage, Springer Vieweg, 2016.
- Ratz, D.; Schulmeister-Zimolong, D.; Seese, D.G.; Wiesenberger, J.: **Grundkurs Programmieren in Java**, 9. Auflage, Hanser, 2024.

Festlegung der Sprache Java – keine Programmierereinführung(!):

- Gosling, J.; Joy, B.; Steele, G. et al.: **The Java Language Specification**, <https://docs.oracle.com/javase/specs/>

Vorlesung

2×4-„stündige“ Vorlesung:

- Mittwoch, 17:15–20:30 Uhr, BT7 10.01
- Donnerstag, 16:15–19:30 Uhr, BT7 04.60

Präsentation liegt i.d.R. am Tag vor Vorlesung in Moodle

Empfehlung: Präsentation zur Einstimmung vor Vorlesung ansehen

Praktikum

4-„stündiges“ Praktikum, 14-tägig (Termine gemäß myHAW):

- **Hilfe** bei Bearbeitung aktueller Übungsaufgaben/Themen
- **Besprechung von Lösungen** zu vorherigen Übungsaufgaben (**Präsentation und Diskussion** durch Teilnehmer:innen)
- Erfüllen der Prüfungsvoraussetzung (dazu gleich)

Empfehlung: **Vorbereitet** zu den Praktika erscheinen:

- **vorherige** Aufgaben sollten bearbeitet worden sein und ggf. präsentiert werden können
- **aktuelle** Aufgaben sollten zumindest angedacht worden sein (falls keine Fragen $\Rightarrow$ keine Hilfe möglich)

Aufgaben

Aufgabenblätter zu dem Vorlesungsthema einer **Semesterwoche X**

- stehen **vor den Vorlesungen der Woche X** in Moodle bereit, sind
- bei **Hilfestellung in Praktika der Wochen X+1 und X+2**
- **bis Freitag der Woche X+2** zu bearbeiten und **einzureichen**
- werden **in den Praktika der Wochen X+3 und X+4** besprochen.

z.B. für Übungsblatt 01:

- Ausgabe: Mittwoch, 15.04. (heute)
- Bearbeitungsende: Freitag, 01.05.
- Besprechung: Di. 05.05. bis Do. ~~14.05.~~ (Feiertag) 21.05.

Hinweis: Moodle-**Forum Inhalte** für **Fragen zu Aufgaben**

Prüfungsvoraussetzung

Zulassung zur Prüfung bei:

- Einreichung von „funktionierenden“ Lösungen zu min. 50% der einzureichenden Aufgaben
- Teilnahme an min. 80%, also 5 der 6 Praktikumstermine
- Präsentation der Lösungen zu min. 2 Aufgaben (auf Zuruf)

Prüfung

Programmiermethodik 1:

Modus: 90-minütige schriftliche **Klausur** (auf Papier)

Aufgaben: Theorie, Methodik und Programmierung

Hilfsmittel: Stift, beidseitig handbeschriebenes DIN-A4-Blatt

Anforderung: bestanden, wenn 50% der Punkte erreicht

Vorbereitung: **Probeklausuren** vor Semesterende in Moodle

Programmiertechnik:

Modus: 90-minütige praktische **Klausur** (am Pool-Rechner)

Aufgaben: Programmieraufgaben

Hilfsmittel: „Spickzettel“ (s.o.)

Anforderung: bestanden, wenn 50% der Punkte erreicht

Kontext im Curriculum

im Curriculum direkt auf PM1-PT aufbauende Kette von Modulen:

2. Semester: Programmiermethodik II

2. Semester: Datenbanken

3. Semester: Algorithmen & Datenstrukturen

3. Semester: Software Engineering I

3. Semester: Betriebssysteme

3. Semester: Intelligente Sensorsysteme

USW.

Lernziele

- **Prinzipien** der Programmierung (Theorie)
- „**Handwerk**“ der Programmierung (Praxis)
- (insb. für Erstsemester:) **Arbeitsweise** im Studium

Vorgehensweise

- 1 „Crashkurs“ (ca. 4 Wochen): elementare Techniken, einfache aber sinnvolle Programme schreiben
- 2 Grundlagen: strukturierte, umfassende Einführung
- 3 Vertiefung: Gestaltung komplexer Software